ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

A Practical Knowledge Base for Retrieval-Augmented Generation

---

Core concepts • Algorithms • Neural networks • NLP • Generative AI • Responsible practice

Prepared as an original educational reference document for a local RAG chatbot project.
Version 1.0 | November 2026

Contents

1. Foundations of Artificial Intelligence

2. Machine Learning: Tasks, Data and Evaluation

3. Supervised Learning Algorithms

4. Unsupervised Learning and Representation

5. Deep Learning and Neural Networks

6. Natural Language Processing

7. Transformers and Large Language Models

8. Embeddings, Vector Search and RAG

9. Building a Reliable RAG Pipeline

10. Evaluation, Safety and Responsible AI

11. Deployment, Monitoring and Maintenance

12. Practical Case Studies and Glossary

Appendix A. RAG-specific facts for retrieval testing

Appendix B. Suggested questions and source notes

1. Foundations of Artificial Intelligence

Artificial intelligence (AI) is the field of designing computational systems that perform tasks commonly associated with human intelligence. These tasks include perception, language understanding, reasoning, planning, learning and decision support. AI is not a single algorithm; it is a broad collection of methods, systems and disciplines.

1.1 What makes a system intelligent?

An AI system receives inputs from an environment, processes information and produces outputs or actions. A spam filter classifies a message, a navigation system proposes a route, and a speech assistant maps audio to language and an action. The system may use explicit rules, learned statistical patterns, or a combination of both.

Intelligence in software is usually defined operationally: how well does the system achieve a specified goal under stated conditions? A narrow system can be highly capable in one task while having no general understanding outside that task. A chess engine, for example, can search chess positions effectively but cannot automatically transfer that skill to medical diagnosis.

• An agent observes its environment through inputs or sensors.

• A policy or decision process maps observations to actions.

• Performance should be assessed against a clearly defined objective and operating context.

1.2 Symbolic AI and data-driven AI

Symbolic AI represents knowledge through explicit symbols, rules, logic and structured relationships. Rule-based expert systems are a classic example: a set of human-authored conditions leads to a conclusion. Such systems can be interpretable, but they may become difficult to maintain when rules are numerous or exceptions are common.

Data-driven AI learns patterns from examples. Instead of writing every decision rule, developers provide training data and an objective function. The learning algorithm adjusts model parameters to reduce errors. Modern machine learning and deep learning are largely data-driven approaches. Hybrid systems combine learned components with rules, constraints or knowledge graphs.

• Use explicit rules when requirements are stable and precisely expressible.

• Use learned models when patterns are complex or difficult to enumerate.

• Use hybrid designs when learned flexibility and explicit constraints are both important.

1.3 AI, machine learning and deep learning

AI is the broadest term. Machine learning (ML) is a subfield of AI in which systems improve performance by learning from data. Deep learning is a subfield of ML that uses neural networks with multiple layers to learn representations. The relationship is nested: deep learning is part of machine learning, and machine learning is part of AI.

A conventional program applies instructions written by a developer to input data. A machine-learning workflow uses examples to estimate a model. The resulting model is then used to make predictions on new inputs. The model does not become universally intelligent; it learns statistical regularities relevant to its training objective and data.

• AI: the overall discipline of intelligent computational systems.

• ML: algorithms that learn patterns from examples.

• Deep learning: multi-layer neural-network methods for representation learning.

1.4 Common application areas

AI is used in image classification, speech recognition, translation, recommendation, fraud detection, forecasting, robotics, scientific discovery and document analysis. Each application has different requirements for latency, accuracy, explainability, privacy and the cost of mistakes.

A useful project begins with a problem statement rather than a fashionable model. Define who uses the system, what decision it supports, what data is available, what errors matter, and how success will be measured. This prevents a technically impressive model from solving the wrong problem.

• Perception: interpreting images, audio, video and sensor streams.

• Language: searching, classifying, summarizing and generating text.

• Prediction: estimating future events or unknown values.

• Decision support: ranking options while keeping people responsible for consequential choices.

2. Machine Learning: Tasks, Data and Evaluation

Machine learning converts examples into a predictive or descriptive model. A sound workflow treats data quality, evaluation and deployment as first-class engineering concerns rather than as steps that happen after model training.

2.1 Main learning settings

In supervised learning, each training example includes an input and a target label or value. Classification predicts a category, such as whether a transaction is suspicious. Regression predicts a numerical quantity, such as expected delivery time.

In unsupervised learning, examples do not have target labels. The system looks for structure such as clusters, low-dimensional representations or unusual observations. Self-supervised learning creates training targets from the data itself; it is widely used to pretrain language and vision models.

Reinforcement learning studies an agent that takes actions and receives rewards or penalties over time. The agent learns a policy intended to maximize cumulative reward. It is useful in sequential decision settings, but reward design and safe exploration can be challenging.

• Supervised: labeled examples; classification and regression.

• Unsupervised: structure discovery without supplied labels.

• Self-supervised: targets derived from the input data.

• Reinforcement: actions, states and reward feedback.

2.2 Data preparation

Data preparation includes collecting relevant examples, checking permissions, removing duplicates, resolving inconsistent formats, handling missing values and creating useful features. A dataset should reflect the population and conditions in which the model will operate. A large dataset can still be poor if it contains systematic gaps or incorrect labels.

Data leakage occurs when information unavailable at prediction time enters training or evaluation. For example, a model predicting loan default should not use a field that is only populated after default occurs. Leakage can create excellent test scores that collapse in production.

Split data into training, validation and test sets. Training data fits parameters, validation data supports model selection, and the test set provides a final estimate. For time-dependent data, split chronologically where appropriate rather than randomly mixing past and future.

• Inspect distributions, missingness, duplicates and label quality.

• Keep a clear record of data origin, permissions and transformations.

• Fit preprocessing steps only on training data, then apply them to validation and test data.

• Use group-aware or time-aware splitting when records are related.

2.3 Generalization and overfitting

A model generalizes when it performs well on previously unseen examples drawn from the intended operating environment. Overfitting occurs when a model learns accidental details of training examples and performs poorly on new data. Underfitting occurs when a model is too simple, poorly trained or insufficiently informative to capture the relevant pattern.

Regularization discourages overly complex solutions. Common techniques include L1 or L2 penalties, dropout in neural networks, early stopping and data augmentation. More data can help when it is relevant, correctly labeled and representative; simply increasing volume does not guarantee better performance.

• Compare training and validation performance.

• Use regularization and appropriate model complexity.

• Investigate distribution shifts rather than assuming the test set represents every future case.

2.4 Evaluation metrics

Accuracy is the proportion of predictions that are correct. It can be misleading when classes are imbalanced. Precision measures the fraction of predicted positives that are truly positive. Recall measures the fraction of actual positives that the model finds. The F1 score is the harmonic mean of precision and recall.

For regression, mean absolute error (MAE) averages absolute prediction errors, while mean squared error (MSE) gives larger errors more weight. Root mean squared error is the square root of MSE and is expressed in the target's units. Metric choice should reflect the cost of different errors in the application.

• Classification: accuracy, precision, recall, F1 and confusion matrix.

• Ranking: mean reciprocal rank, precision at k and normalized discounted cumulative gain.

• Regression: MAE, MSE and root mean squared error.

• Probability quality: calibration and suitable probabilistic scoring rules.

3. Supervised Learning Algorithms

Supervised algorithms learn a mapping from input features to known targets. Different algorithms make different assumptions about the shape of that mapping, the amount of data required and the trade-off between interpretability and flexibility.

3.1 Linear and logistic regression

Linear regression predicts a continuous value as a weighted sum of input features plus an intercept. It is a useful baseline because its parameters are relatively easy to inspect. Its limitations include sensitivity to outliers and difficulty representing complex nonlinear relationships without feature transformations.

Logistic regression is commonly used for binary classification. It maps a weighted sum of features through a logistic function to produce a value between zero and one. A threshold converts that probability-like score into a class decision. The threshold should be chosen based on the application rather than assumed to be 0.5 in every case.

• Standardize numeric features when scales differ substantially.

• Check correlated predictors and influential outliers.

• Evaluate probability calibration when scores are used for decisions.

3.2 Decision trees and random forests

A decision tree repeatedly divides the feature space using questions about feature values. Each leaf provides a prediction. Trees can represent nonlinear relationships and interactions, but a deep tree may memorize training examples. Pruning, depth limits and minimum leaf sizes can reduce this risk.

A random forest trains many trees on bootstrapped samples and considers subsets of features when splitting. Averaging or voting across trees reduces the variance of an individual tree. Random forests are robust baselines for many tabular problems, although large forests can be less transparent than a single tree.

• Inspect tree depth and leaf sizes.

• Use feature importance carefully; it is not proof of causality.

• Compare performance with a simple baseline.

3.3 Support vector machines and k-nearest neighbors

A support vector machine (SVM) seeks a decision boundary with a large margin between classes. Kernel functions allow an SVM to represent nonlinear boundaries without explicitly constructing every transformed feature. Scaling is important, and training can become expensive on very large datasets.

k-nearest neighbors (k-NN) predicts from nearby training examples. For classification it may use a majority vote; for regression it may average neighbor targets. The meaning of 'nearby' depends on the distance measure and feature scaling. Irrelevant or high-dimensional features can make distances less useful.

• SVMs can work well on moderate-sized, carefully prepared datasets.

• k-NN is simple but can be costly at prediction time.

• Choose distance metrics and feature scaling deliberately.

3.4 Gradient boosting

Gradient boosting builds an ensemble sequentially. Each new weak learner is trained to improve the errors made by the current ensemble, often by following the gradient of a loss function. Implementations such as gradient-boosted decision trees are popular for structured tabular data.

Boosting can achieve strong predictive performance, but it requires careful tuning. Learning rate, number of trees, tree depth and subsampling affect the result. Early stopping based on validation performance can prevent unnecessary complexity.

• Tune learning rate together with the number of estimators.

• Use validation data for early stopping.

• Compare against random forests and simpler baselines.

3.5 Choosing a supervised model

Model selection should consider predictive quality, interpretability, data volume, inference latency, memory, maintenance and fairness. A simpler model that is easier to explain and monitor may be preferable to a marginally more accurate complex model. Establish a baseline before investing in advanced methods.

• Start with a simple baseline.

• Use cross-validation when data volume and independence assumptions allow it.

• Record model versions, features, metrics and configuration for reproducibility.

4. Unsupervised Learning and Representation

Unsupervised methods explore data without relying on manually supplied target labels. They are useful for discovery, compression, anomaly analysis and organizing large collections of examples.

4.1 Clustering

Clustering groups observations according to a similarity or distance rule. K-means assigns each point to the nearest centroid and updates centroids until assignments stabilize. It is computationally efficient but requires a chosen number of clusters and can be affected by scaling, outliers and non-spherical cluster shapes.

Hierarchical clustering builds a tree of nested groups. Density-based methods such as DBSCAN identify dense regions separated by sparse areas and can mark isolated points as noise. No single clustering method is best for every dataset; the result must be checked against domain meaning.

• Scale features when distance comparisons require it.

• Try multiple cluster counts and inspect cluster stability.

• Use domain review to determine whether groups are meaningful.

4.2 Dimensionality reduction

Dimensionality reduction maps data into fewer dimensions while attempting to preserve useful structure. Principal component analysis (PCA) finds orthogonal directions that capture high variance in the data. It is often used for compression, visualization and noise reduction.

Nonlinear visualization techniques can reveal local neighborhoods in high-dimensional data, but a two-dimensional plot should not be treated as a complete representation of the original geometry. A visually separated plot does not by itself prove that the groups are semantically distinct.

• PCA is linear and its components can be inspected.

• Visualization methods are useful for exploration, not as sole evidence.

• Fit transformations on training data when building predictive pipelines.

4.3 Anomaly detection

Anomaly detection identifies observations that differ from expected patterns. Applications include unusual network activity, equipment faults and suspicious transactions. Methods include statistical thresholds, isolation forests, one-class SVMs and reconstruction-error approaches.

An anomaly is not automatically harmful or incorrect. Rare events may be legitimate, and a system may generate many false alarms when the normal population is diverse. Human review, alert prioritization and feedback loops are often important.

• Define what counts as an anomaly in the application.

• Measure false alarms and missed events.

• Keep a review process for uncertain or high-impact alerts.

4.4 Feature learning and embeddings

A representation converts an object into a numerical form that preserves information useful for a task. An embedding is a vector representation in which related items are intended to occupy nearby regions of a vector space. Text, images, audio and users can all be represented by embeddings.

Embedding quality depends on the training objective, domain, language, input length and intended similarity. Two passages may be topically related but answer different questions. Retrieval systems therefore benefit from evaluating actual question-to-passage matches rather than assuming vector proximity always means relevance.

- Use embeddings appropriate to the content type and language.
- Keep model version and vector dimensions consistent in an index.
- Evaluate retrieval using representative queries and judged relevant passages.

5. Deep Learning and Neural Networks

Deep learning uses neural networks with multiple layers to learn increasingly abstract representations. It has been especially influential in computer vision, speech, language and generative modeling.

5.1 The artificial neuron

A basic artificial neuron calculates a weighted sum of inputs, adds a bias and applies an activation function. The weights determine how strongly each input contributes. The activation function introduces nonlinearity, allowing networks to represent relationships that a purely linear stack could not express.

Common activation functions include ReLU, sigmoid and tanh. ReLU returns zero for negative inputs and the input itself for positive values. Sigmoid maps values to the interval between zero and one and is often used for binary output probabilities. The choice depends on the layer and learning objective.

• Inputs are multiplied by learned weights.

• A bias shifts the weighted sum.

• An activation function transforms the result.

5.2 Layers and network architecture

A feed-forward neural network passes information from an input layer through one or more hidden layers to an output layer. The input layer represents features, hidden layers transform them, and the output layer produces the prediction. Architecture includes layer count, width, activation functions and connections.

Convolutional neural networks use local filters and shared weights to detect spatial patterns, especially in images. Recurrent neural networks process sequences while maintaining a state, although transformer architectures have become dominant in many language tasks. Architecture should follow the structure of the data and task.

• CNNs: local spatial pattern processing.

• RNNs: sequential processing with recurrent state.

• Transformers: attention-based interaction between sequence elements.

5.3 Training with backpropagation

Training typically involves a forward pass, loss calculation, backward pass and parameter update. The loss function measures the mismatch between predictions and targets. Backpropagation applies the chain rule to calculate gradients of the loss with respect to network parameters.

An optimizer uses gradients to update parameters. Stochastic gradient descent and Adam are common choices. The learning rate controls update size. If it is too large, training may be unstable; if too small, learning may be slow. Training curves help reveal convergence problems and overfitting.

• Forward pass: calculate predictions.

• Loss: quantify prediction error.

• Backpropagation: calculate gradients.

• Optimization: update weights and biases.

5.4 Regularization and normalization

Dropout randomly disables a fraction of activations during training, reducing reliance on particular pathways. Weight decay penalizes large parameter values. Data augmentation creates modified training examples that preserve labels, such as cropping or flipping suitable images.

Normalization methods help stabilize activation distributions and training. Batch normalization uses statistics from mini-batches during training, while layer normalization normalizes within an example and is common in transformer architectures. These methods are not interchangeable in every setting.

5. Deep Learning and Neural Networks

Deep learning uses neural networks with multiple layers to learn representations from data. It is especially influential in image recognition, speech, language and generative modeling.

The artificial neuron

A basic artificial neuron calculates a weighted sum of inputs, adds a bias and applies an activation function. Weights determine how strongly each input contributes. The activation introduces nonlinearity, allowing a network to represent complex relationships.

Common activation functions include ReLU, sigmoid and tanh. ReLU returns zero for negative inputs and the input itself for positive inputs. Sigmoid maps values to a range between zero and one and is often used in binary output layers.

Layers and architectures

A feed-forward network passes information from an input layer through hidden layers to an output layer. Convolutional neural networks use local filters and shared weights to detect spatial patterns, particularly in images. Recurrent neural networks process sequences while maintaining a state. Transformers use attention to combine information across sequence positions.

Architecture should reflect the structure of the data and the task. A model with more layers is not automatically better; it may require more data, computation and careful regularization.

Training and backpropagation

Training typically involves a forward pass, loss calculation, backward pass and parameter update. The loss measures mismatch between predictions and targets. Backpropagation applies the chain rule to calculate gradients with respect to network parameters.

An optimizer uses gradients to update weights. Stochastic gradient descent and Adam are common choices. The learning rate controls update size: a rate that is too large can make training unstable, while a rate that is too small can make learning slow.

Regularization and limitations

Dropout randomly disables some activations during training. Weight decay penalizes large parameter values. Data augmentation creates modified examples while preserving their labels. Validation curves help detect overfitting.

Deep networks can require substantial data, compute and energy. Their predictions may be difficult to explain, and performance can degrade when operating conditions differ from training data. A confident output is not necessarily correct.

6. Natural Language Processing

NLP develops computational methods for processing human language. Text is ambiguous and context-dependent, so systems must preserve useful context while transforming text into representations a model can process.

Tokenization and preprocessing

Preprocessing may include normalization, sentence segmentation and tokenization. Tokenization divides text into units used by a model. A token may be a word, subword or character, depending on the tokenizer.

Subword tokenization helps models handle rare words by composing them from smaller units. Character count and token count are not the same. Context limits are generally measured in tokens, so the tokenizer associated with a model should be used when estimating prompt size.

Common NLP tasks

Text classification assigns categories such as topic, sentiment or intent. Named entity recognition identifies spans such as people, organizations, locations and dates. Information extraction converts unstructured text into structured fields.

Machine translation maps text between languages. Summarization creates a shorter representation of a source. Question answering produces an answer using model knowledge, supplied context or both. Search and retrieval identify passages likely to help answer a query.

Language models and evaluation

A language model estimates the likelihood of token sequences. Autoregressive models predict the next token from previous tokens. Masked language models learn to predict hidden tokens from surrounding context. Pretraining learns broad patterns; fine-tuning adapts a model using additional examples.

Evaluation should match the task. Classification may use precision and recall. Summarization should be checked for coverage and factual consistency. A fluent output can still omit or distort important information, so human review is useful for nuanced tasks.

7. Transformers and Large Language Models

Transformers are neural network architectures built around attention mechanisms. They allow information from different token positions to be combined and are a foundation for many modern language systems.

Self-attention

Self-attention computes relationships among tokens. Each token is projected into query, key and value representations. Similarity between queries and keys determines how information from values is combined. This lets a token representation incorporate information from other positions.

Multi-head attention performs several attention operations in parallel. Different heads can learn different relationships. Positional information is also needed because attention alone does not inherently encode token order.

Transformer configurations

A transformer block commonly contains an attention sublayer and a feed-forward sublayer, with residual connections and normalization. Encoder-only, decoder-only and encoder-decoder configurations serve different tasks.

Encoder-only models produce contextual representations useful for classification or retrieval. Decoder-only models generate text autoregressively. Encoder-decoder models encode an input and generate an output, a structure suited to translation and other sequence transformation tasks.

LLMs, prompting and limitations

A large language model is trained on large-scale data with a language modeling objective and typically contains many parameters. It can draft, explain, summarize and answer questions. Its output reflects learned patterns and the context supplied at inference time.

LLMs can hallucinate: they may produce plausible statements that are unsupported or incorrect. A prompt should state the task, relevant context and output requirements. A context window is the token capacity available for one request. Retrieval can select relevant passages instead of placing an entire document collection into every prompt.

8. Embeddings, Vector Search and RAG

Retrieval-Augmented Generation combines information retrieval with language generation. It is useful when answers should reflect a defined collection of private, specialized or frequently updated documents.

Embeddings

An embedding model maps text into a fixed-length numerical vector. Similarity may be measured with cosine similarity, dot product or Euclidean distance. The appropriate metric depends on the embedding model and whether vectors are normalized.

Document and query embeddings must be compatible. Changing the embedding model can change vector dimensions and the geometry of the vector space. Record the model name, version, dimensions and preprocessing choices so that queries are encoded consistently.

Vector databases

A vector database stores embeddings and supports similarity search. A record commonly contains an identifier, vector, text or document reference, and metadata such as filename, page, section or date.

Chroma is an open-source vector database that can run locally. Pinecone is a managed vector database service. A local Chroma collection is convenient for a learning project because it avoids the need for a hosted account.

Retrieval and ranking

A retrieval pipeline embeds a user question and searches for nearby document vectors. The top-k results are candidate passages. A small k may omit necessary context; a large k may introduce irrelevant chunks. Retrieval should be inspected using representative questions.

A reranker scores query-passage pairs more directly and can reorder candidates. Hybrid retrieval combines semantic search with keyword search, which can help when exact identifiers or technical terms matter. Metadata filtering restricts search to a source, date or category.

What RAG changes

In a plain LLM call, the model receives the prompt and relies on learned parameters plus the conversation. In a RAG call, the application first retrieves passages from an external collection and includes them in the prompt. The answer can then be grounded in retrieved content.

RAG does not guarantee correctness. If retrieval misses the right passage or supplies outdated or contradictory chunks, the answer may still be wrong. A strong system evaluates retrieval, instructs the model to use evidence and provides source references.

9. Building a Reliable RAG Pipeline

A reliable RAG application is a data pipeline, a search system and a generation interface. Each stage should be testable independently so failures can be diagnosed.

Ingestion and extraction

Ingestion reads source files and extracts text. PDF extraction may be affected by scanned pages, multi-column layouts, tables, headers and footers. Scanned PDFs may require optical character recognition. Record the original filename, page number and section heading where possible.

Keep an ingestion log with file counts, extraction warnings and chunk totals. Validate that each source produces meaningful text. Empty or garbled extraction can make a document appear to be indexed when it is not actually searchable.

Chunking strategy

Chunking divides long documents into smaller passages. A single huge chunk is difficult to retrieve precisely, while tiny fragments may lose context. A common starting range is approximately 300–800 tokens per chunk, with overlap between adjacent chunks. The best setting depends on document structure and the embedding model.

Overlap preserves context that crosses a chunk boundary. Excessive overlap creates duplicate content and increases retrieval noise. Prefer headings, paragraphs and section boundaries. Inspect sample chunks manually and avoid splitting tables or numbered procedures in ways that destroy relationships.

Prompt construction

A grounded generation prompt identifies retrieved passages as reference material, states the user's question and defines how to behave when evidence is insufficient. It should instruct the model not to invent missing facts and, where supported, to cite the source passage.

Retrieved documents are data, not instructions that override system rules. A document may contain malicious or irrelevant text. Treat retrieved content as untrusted evidence and do not allow it to change system-level behavior.

End-to-end flow and debugging

Offline ingestion: files → extraction → chunking → embeddings → vector database. Online querying: question → query embedding → retrieval → prompt construction → LLM → answer. Keep ingestion and query execution separate so documents are not re-embedded for every question.

Debug each stage in order: print extracted text, inspect chunks, query the vector store, inspect retrieved passages and only then test generation. Common failures include wrong collection names, incompatible embedding models, overly restrictive metadata filters and poor PDF extraction.

10. Evaluation, Safety and Responsible AI

Evaluation asks whether a system works for its intended users and operating conditions. Responsible AI also considers fairness, privacy, transparency, security and the consequences of errors.

Retrieval evaluation

Recall at k asks whether a relevant passage appears within the top k results. Precision at k measures how many of the top k results are relevant. Mean reciprocal rank rewards systems that place the first relevant result near the top.

Create an evaluation set with questions, expected source passages and relevance judgments. Include direct questions, paraphrases, questions requiring multiple passages and questions that are not answered by the collection.

Generation evaluation

Generated answers can be evaluated for correctness, relevance, completeness, groundedness and clarity. Groundedness asks whether claims are supported by retrieved context. A response should not score highly merely because it sounds fluent.

Automated metrics help compare versions, but human review remains valuable. Keep examples of unsupported claims, missing citations and incorrect abstentions as regression tests.

Privacy, security and fairness

Documents may contain personal, confidential or copyrighted material. Confirm authorization before ingestion. Restrict access to the vector database and protect backups. Do not commit private documents, API keys or local database files to a public repository unless intentionally public and licensed for sharing.

Prompt injection can appear in user input or retrieved documents. Treat retrieved text as untrusted data. Apply access controls before retrieval so users cannot retrieve documents they are not permitted to see. Evaluate performance across relevant groups when lawful and appropriate.

Human oversight

Explain what the system does, what sources it uses and where it may fail. For consequential decisions, provide human oversight and a way to challenge or correct outputs. AI should support accountable decision-making rather than obscure responsibility.

When evidence is missing, the system should say so. An abstention is often safer and more useful than a confident guess. Source citations help users verify important claims.

11. Deployment, Monitoring and Maintenance

A prototype becomes useful when it can be run consistently, monitored and updated. Deployment planning should address reliability, cost, access control and document lifecycle.

Interface and reproducibility

A simple interface can let users enter a question, view an answer and inspect retrieved source passages. Streamlit is a Python framework suitable for a lightweight prototype. A command-line interface is also useful during development because it exposes intermediate results clearly.

Use a requirements file to record Python dependencies. Store chunk size, overlap, collection name and model identifier in configuration. Keep secrets outside source control and provide a .env.example file with placeholder values only.

Monitoring and updates

Monitor ingestion failures, retrieval latency, generation latency, token usage, error rates and user feedback. Review retrieved passages to see whether search quality changes as the collection grows. Monitor document freshness and remove or supersede outdated sources through a controlled process.

Changes to the embedding model or chunking strategy may require rebuilding the index. Keep an index version and rollback plan. A successful deployment is not the end of evaluation; it begins ongoing observation.

Scaling and cost

A local Chroma database is convenient for development and small demonstrations. Larger multi-user systems may need managed storage, authentication, backups, replication and operational monitoring. Scale according to actual requirements rather than adding infrastructure prematurely.

Generation costs can be managed through prompt-size limits, caching and appropriate model selection. Do not reduce retrieval quality blindly to reduce cost; evaluate trade-offs using a fixed test set.

12. Practical Case Studies and Glossary

These examples show how retrieval-based systems can support document-heavy workflows. They are illustrative designs, not claims that a model should make consequential decisions without human accountability.

University policy assistant

A university department indexes policy documents, course handbooks and approved procedures. A RAG assistant retrieves relevant sections when a student asks about a process. It cites the source and page so the student can verify the answer.

Important design choices include document freshness, access control, page-level citations and reliable abstention. A fluent answer without a source is not sufficient for policy questions.

Equipment maintenance assistant

A maintenance team indexes manuals and troubleshooting guides. A technician describes an error code and the assistant retrieves the relevant procedure. The system should show the manual version and warn when the document is outdated.

Tests should include exact error codes, paraphrased descriptions and questions whose answers are absent from the manuals. Safety-critical work must follow approved procedures and qualified human judgment.

Research literature explorer

A researcher indexes papers and asks for comparisons of methods, datasets and limitations. The system retrieves passages from multiple papers and produces a structured summary with citations. It should distinguish what each paper reports from conclusions inferred across papers.

Evaluation should check whether the correct paper is found, numerical details are preserved, claims are attributed accurately and references are not fabricated.

Glossary

Artificial intelligence: the field of building systems that perform tasks associated with intelligent behavior.

Algorithm: a defined procedure for transforming inputs into outputs.

Embedding: a numerical vector representation of an item.

Chunk: a segment of a document used as a retrieval unit.

Vector database: a system that stores vectors and supports similarity search.

Reranker: a model that reorders candidate passages by relevance.

Context window: the token capacity available to a model for one request.

Hallucination: generated content that is unsupported or incorrect.

Grounded answer: an answer whose claims are supported by supplied evidence.

Data leakage: use of information during training that would not be available at prediction time.

Appendix A. RAG-specific facts for retrieval testing

This appendix contains a distinctive reference profile for testing whether a RAG application retrieves document-specific facts. The profile is fictional and exists only as a controlled test fixture; it is not a real organization or product specification.

Northstar Learning Lab — fictional test profile

The fictional Northstar Learning Lab maintains a sample knowledge collection called the Aurora Set. The Aurora Set contains 36 approved learning notes, organized into six subject groups. Each subject group contains six notes. The collection identifier used in this example is NSL-AURORA-36.

The lab's sample ingestion policy assigns the label 'bluejay' to documents that have passed both text extraction review and source metadata validation. Documents that have been extracted but not reviewed receive the label 'amber'. These labels are invented for this educational test fixture.

The sample chunking configuration is 420 tokens per chunk with an overlap of 60 tokens. Its retrieval demonstration returns the top 4 candidate chunks. These values are specific to this fictional profile and are not universal RAG recommendations.

Controlled question-answer facts

The Aurora Set has 36 approved notes and six subject groups, with six notes per group. Its collection identifier is NSL-AURORA-36. The reviewed-document label is bluejay; the pending-review label is amber.

The fictional chunk size is 420 tokens, and the overlap is 60 tokens. The retrieval count is top 4 chunks. The sample evaluation contains 24 questions: 16 answerable and 8 unanswerable. Its acceptance target is at least 20 supported or correctly abstained responses out of 24.

These values are intentionally arbitrary. A general-purpose LLM would not be expected to know them before the profile is provided. The RAG system should retrieve this appendix and answer from its content.

Why this appendix matters

Ask a plain LLM for the Aurora Set's collection identifier without supplying the document. It should not know the answer and should not be encouraged to guess. Then ask the same question through RAG. The application should retrieve this appendix and return NSL-AURORA-36 with the source filename and page.

Repeat the test for the bluejay label, chunk overlap and evaluation-set size. Also ask for the fictional lab's office address. The correct behavior is to say that the supplied document does not specify an address. This checks whether the system can abstain instead of inventing details.

Appendix B. Suggested questions and source notes

Use these questions as a starter evaluation set. Add more after inspecting the actual chunks produced by the ingestion pipeline.

General concept questions

What is the difference between AI, machine learning and deep learning?

Why can accuracy be misleading for imbalanced classification?

What is data leakage, and how can it be prevented?

How does self-attention combine information from tokens?

What is the purpose of overlap in document chunking?

What is the difference between a vector database and a language model?

Why should retrieval quality be tested separately from generation quality?

When should a RAG system abstain from answering?

Document-specific questions

What is the fictional Aurora Set collection identifier?

Which label identifies documents that passed extraction review and metadata validation?

How many approved notes are in the fictional Aurora Set?

What chunk size and overlap are specified in the Northstar profile?

How many chunks does the fictional profile retrieve?

How many unanswerable questions are included in its evaluation set?

What is the acceptance target for the sample evaluation?

What does the document say about the fictional lab's office address?

Test procedure and source notes

First, run each document-specific question against a plain LLM call without the PDF context. Save the response and note whether the model admits that it lacks the information.

Next, run the same question through the RAG application. Save the retrieved chunk text, source filename, page number and final answer. The answer should match the appendix for fixed facts.

Finally, ask an unanswerable question. A reliable system should not invent an address or other absent fact. Include all three outcomes in the README: plain LLM, RAG with retrieved evidence and RAG abstention.

This guide is original educational material prepared for a learning project. The Northstar Learning Lab and Aurora Set are fictional test fixtures. When adding external documents to a public repository, check each source's license and attribution requirements.